

sse_challenge_1_Surname1Surname2

April 2, 2026

1 UE “Embedded Systems Security” (2025-2026) – Part 1

1.1 Introduction

This is the first challenge of the evaluated TP.

The **Context** section presents in detail the challenge. Several **questions** will guide you through the solution of the challenge.

The solution to this challenge will provide you the password to access to the second challenge.

Read carefully each section. Do not hesitate to contact the supervisor should you have any doubt or question.

1.1.1 What Material You Can Use

You can use: 1. All the code you developed during the TPs; 2. You can refer to the slides presented during the CMs; 3. You can refer to any other source (textbooks, research papers, blog posts, stackoverflow, ...); 4. You may use LLMs/AI coding assistants only to help you with coding.

1.1.2 What to Deliver

To have the challenges evaluated, you have to deliver the Jupyter notebooks completed with your answers.

You **MUST** deliver the notebooks before the end of the last TP. Deliver the notebooks (saved in a compressed format, such as TAR, GZ, ZIP, RAR, ...) on the moodle.

The jupyter notebooks must be named as ‘sse_challenge_X_Surname1Surname2’, where: * X: it is the challenge number; * Surname1 and Surname2 are the surnames of the binome.

Rename your compressed archive as **sse_challenges_Surname1Surname2**.

If you are working alone, just put your surname.

1.2 Context

You are assessing the attack surface for a client providing a FPGA-as-a-Service platform. According to the design requirements given by your client, the access should be provided only to authorised users.

Authorisation checks happen through the use of an authentication protocol based on FIDO (*Fast Identity Online*):

Authentication over an insecure channel happens using public-key cryptography. The user's device registers the user to a server by registering a public key. To authenticate the user, the device signs a challenge from the server using the private key that it holds. The keys on the device are unlocked by a local user gesture such as a biometric or pressing a button.

In the documentation, the client refers to the authentication device as “AuDe”. After a rigorous testing of the software stack composing the authentication protocol, it is time to test the root of trust: AuDe.

To gain access to the cloud service, one has to correctly *sign* a message sent from one of the service's servers. As specified in the documentation, the signing of the message is done by AuDe, which hardcodes the private key. Of course, one may try invasive techniques, like Fault Injection or Micro-probing, to recover the private key; but before attempting advanced techniques, it is usefull to assess the security surface of the device against simpler and effective attacks: the physical side-channel attacks.

The client hands you the C implementation of the code responsible to sign a message with the private key:

```
// Function to compute modular exponentiation ( $a^b \bmod m$ )
uint32_t mod_exp(uint32_t message, uint32_t private_key, uint32_t mod) {
    uint64_t result = 1;
    message = message % mod;
    while (private_key > 0) {
        if (private_key % 2 == 1) {
            result = (result * message) % mod;
        }
        private_key = private_key >> 1;
        message = (message * message) % mod;
    }
    return result;
}
```

The `mod_exp` sign as `message` with the `private_key` by computing the modular exponentiation of the message with the private key. Let us start the hardware security audit.

1.2.1 Question 1

From the `mod_exp` piece of code, do you remark any potential vulnerability that may allow an attacker to deduce some information on the private key? Write in the cell below an brief but exhaustive explanation for your client.

Your Answer

[]:

1.2.2 Question 2

You have recorded a power consumption trace while AuDe performs the signing of a message. Let us check whether the power consumption profile confirms your hypothesis in **Question 1**.

```
[ ]: import matplotlib.pyplot as plt

trace = # load the trace

assert trace.ndim == 1

plt.plot(trace)
plt.show()
```

Does the power shape confirms your hypothesis in **Question 1**? Provide a brief but exhaustive answer. You can resort on images (for instance, use matplotlib to plot part of the trace) to support your explanation.

Your Answer

```
[ ]:
```

1.2.3 Question 3

To prove the severity of the discovered vulnerability, you need to provide a reproducible proof-of-concept attack.

To do this: 1. Briefly describe the type of attack you can run to exploit the identified vulnerability; 2. Provide a description (for instance, a pseudo-code) of how the attack may work; 3. Provide a python implementation of the above description; 4. Recover the private key employed by AuDe.

Your Answer

```
[ ]:
```

1.2.4 Question 4

Now that you have recovered the correct key, try to sign the message associated with the recorded power trace.

If you have used the correct private key, you will have access to the next challenge.

```
[ ]: def sign_message(message: int, private_key: int, mod: int) -> int:
    """
    Compute modular exponentiation: (message^private_key) % mod
    """
    result = 1
    message = message % mod

    while private_key > 0:
        if private_key % 2 == 1:
```

```
        result = (result * message) % mod

    private_key = private_key >> 1
    message = (message * message) % mod

    return result
```

```
[ ]: message = 0x48616c66 # the message associated with the power trace

print(f"Signing message {message} with key {key}")

signed_message = sign_message(key, message, 37665977)

print(f"Signed message: {signed_message}")
print(f"The password to the next challenge: >>SSE_{hex(signed_message)[2:]}")
```

1.2.5 Question 5

Could you outline to the client a solution to the vulnerability that you have spot?

Your Answer

```
[ ]:
```

1.3 End of the Challenge

Save this notebook, rename it as `sse_challenge_1_Surname1Surname2`, where Surname1 and Surname2 are the surnames of the pair that has completed this challenge.